

Seasonal Decomposition

```
import pandas as pd
import numpy as np
import os, pathlib
from pathlib import Path
```

```
block_df = pd.read_parquet('hbblock_df.parquet')
```

```
exp_block_df = compact_to_expanded(
    block_df, timeseries_col='energy_consumption',
    static_cols=['frequency', 'series_length', 'stdorToU', 'Acorn', 'Acorn_grouped'],
    time_varying_cols=['pressure', 'apparentTemperature', 'windSpeed', 'precipType', 'icon', 'humidity', 's
ts_identifier='LCLid')
```

0%| | 0/799 [00:00<?, ?it/s]

```
exp_block_df.head(1)
```

	timestamp	LCLid	energy_consumption	frequency	series_length	stdorToU	Acorn	Acorn_grouped	pressu
0	2012-10-13	MAC000002	0.263	30min	24144	Std	ACORN-A	Affluent	100

Train, Test and Validation Sets

```
test_mask = (exp_block_df.timestamp.dt.year==2014) & (exp_block_df.timestamp.dt.month==2)
validation_mask = (exp_block_df.timestamp.dt.year==2014) & (exp_block_df.timestamp.dt.month==1)

train = exp_block_df[~(test_mask | validation_mask)]
validation = exp_block_df[validation_mask]
test = exp_block_df[test_mask]
train.shape, validation.shape, test.shape
```

((11855088, 15), (595200, 15), (518400, 15))

Baseline

```
train_df = train[['LCLid', 'timestamp', 'energy_consumption', 'frequency']]
validation_df = validation[
    ['LCLid', 'timestamp', 'energy_consumption', 'frequency']]
test_df = test[['LCLid', 'timestamp', 'energy_consumption', 'frequency']]
```

```
train_df.timestamp.min(), train_df.timestamp.max(), \
test_df.timestamp.min(), test_df.timestamp.max(), \
validation_df.timestamp.min(), validation_df.timestamp.max()
```

(Timestamp('2012-01-01 00:00:00'),
Timestamp('2013-12-31 23:30:00'),
Timestamp('2014-02-01 00:00:00'),
Timestamp('2014-02-27 23:30:00'),
Timestamp('2014-01-01 00:00:00'),
Timestamp('2014-01-31 23:30:00'))

```
freq = train_df.iloc[0]['frequency']
timeseries_train = train_df.loc[train_df.LCLid=='MAC000322', ['LCLid', 'timestamp', 'energy_consumption']]
timeseries_validation = validation_df.loc[validation_df.LCLid=='MAC000322', ['LCLid', 'timestamp', 'energy_
timeseries_test = test_df.loc[test_df.LCLid=='MAC000322', ['LCLid', 'timestamp', 'energy_consumption']]
```

```
predictions = pd.concat([timeseries_train, timeseries_validation])
predictions.head(), predictions.shape, predictions.dtypes
```

(
LCLid timestamp energy_consumption
0 MAC000322 2012-03-07 00:00:00 0.125
1 MAC000322 2012-03-07 00:30:00 0.104
2 MAC000322 2012-03-07 01:00:00 0.133
3 MAC000322 2012-03-07 01:30:00 0.145
4 MAC000322 2012-03-07 02:00:00 0.109,
(33408, 3),
LCLid object
timestamp datetime64[ns]
energy_consumption float64
dtype: object)

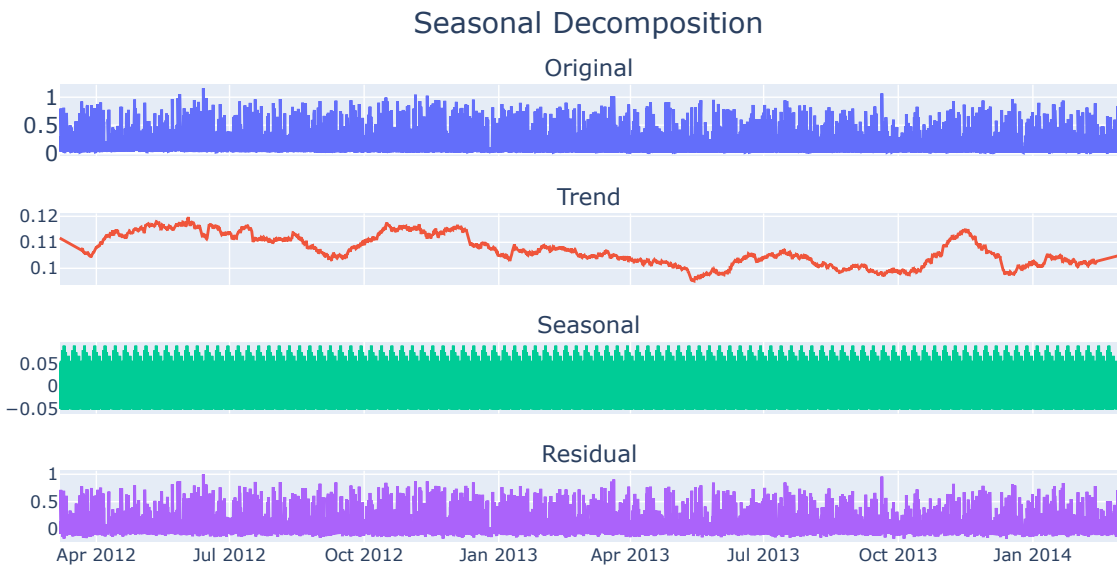
```
ts = SeasonalInterpolation(seasonal_period=48*7).fit_transform(ts_df.energy_consumption.values.reshape(-1,1
```

Seasonal Decomposition

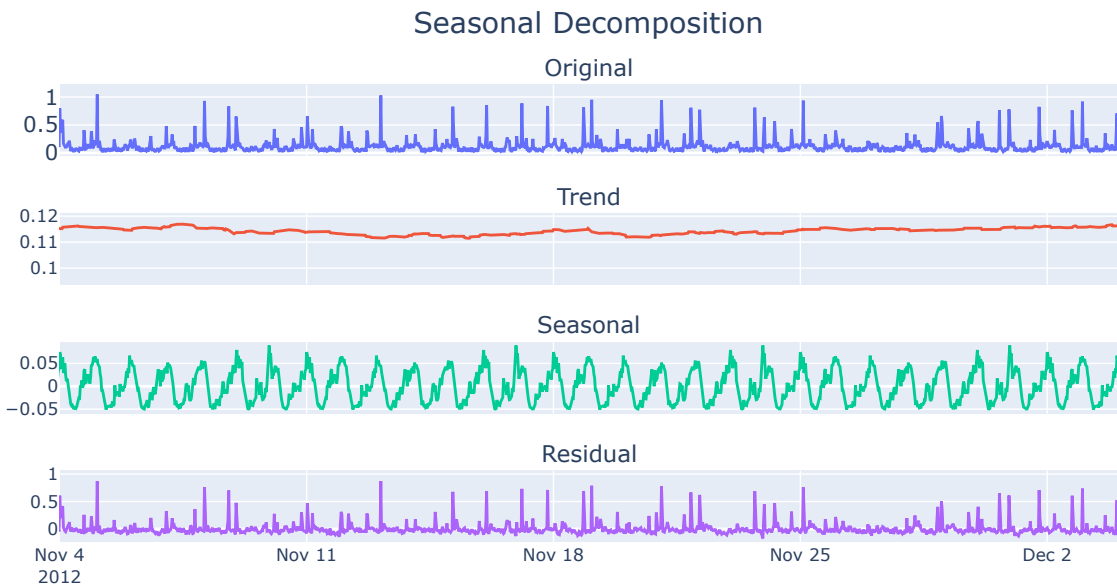
```
from statsmodels.tsa.seasonal import seasonal_decompose

#Does not support missing values, so using imputed ts instead
res = seasonal_decompose(ts, period=7*48, model="additive", extrapolate_trend="freq", filt=np.repeat(1/(30*

fig = decomposition_plot(ts_df.index, res.observed, res.seasonal, res.trend, res.resid)
fig
```



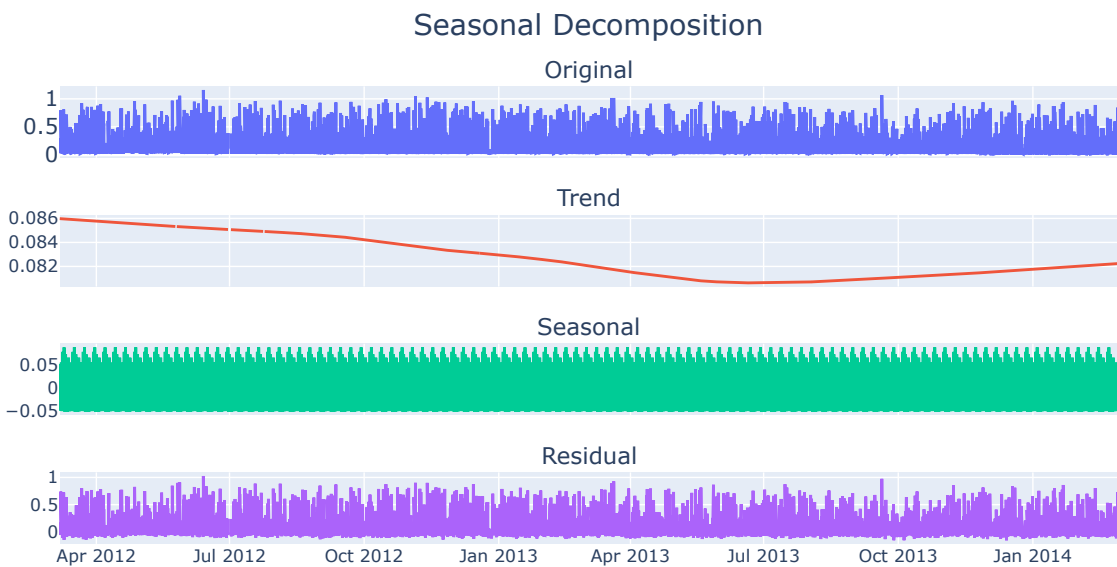
```
#Let's zoom in on a few days to better see the seasonality extracted
fig.update_xaxes(type="date", range=["2012-11-4", "2012-12-4"])
fig #.show(renderer="svg")
```



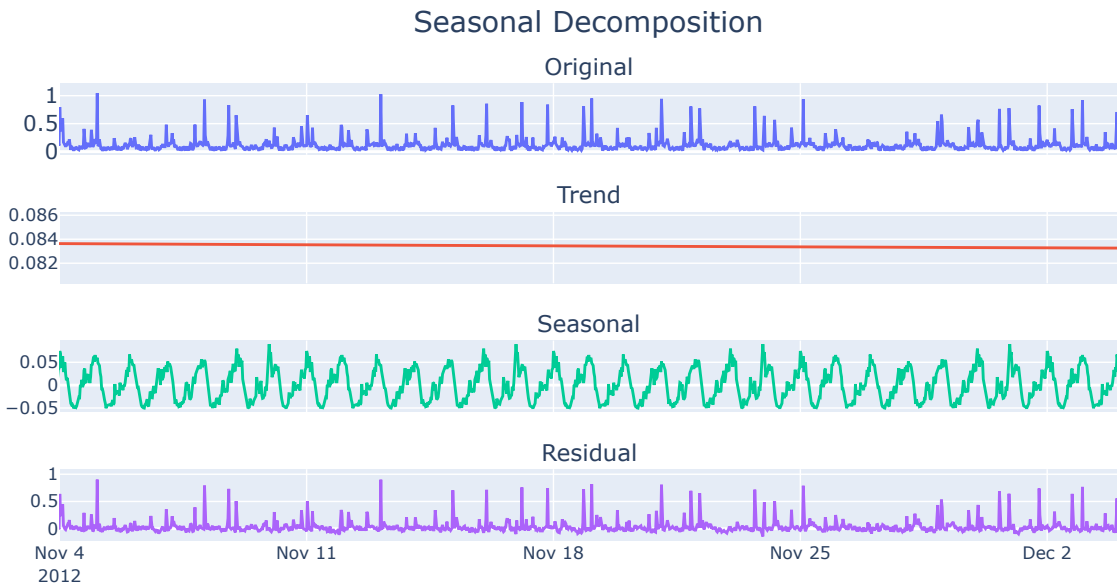
Seasonality and Trend Decomposition using Loess (STL)

```
#Supports missing values and expects a series or dataframe with datetime index
stl = STL(seasonality_period=7*48, model = "additive")
res_new = stl.fit(ts_df.energy_consumption)
```

```
fig = res_new.plot(interactive=True)
fig
```



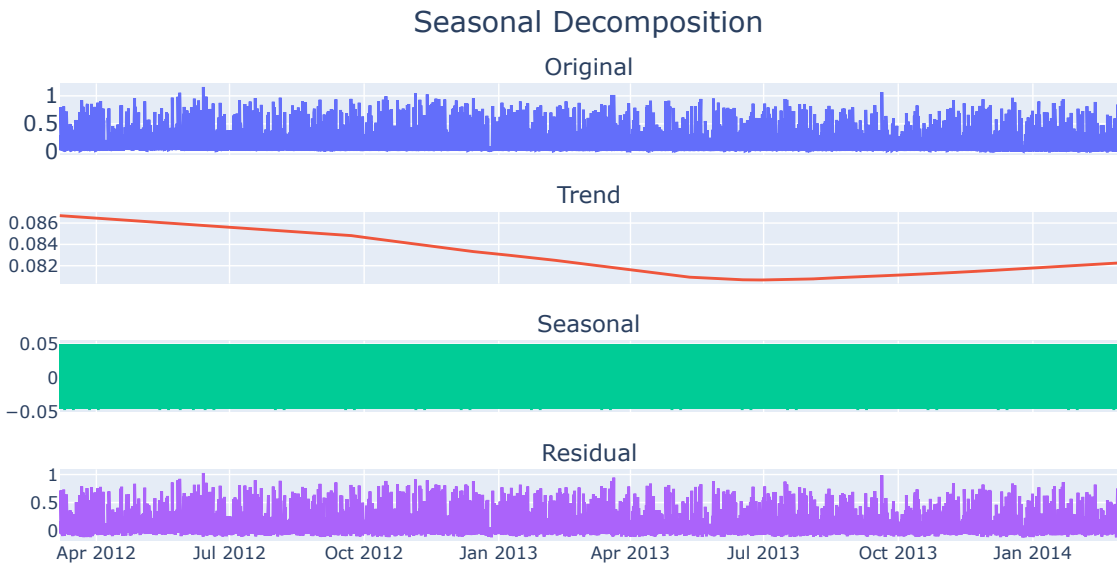
```
#Let's zoom in on a few days to better see the seasonality extracted
fig.update_xaxes(type="date", range=["2012-11-4", "2012-12-4"])
fig
```



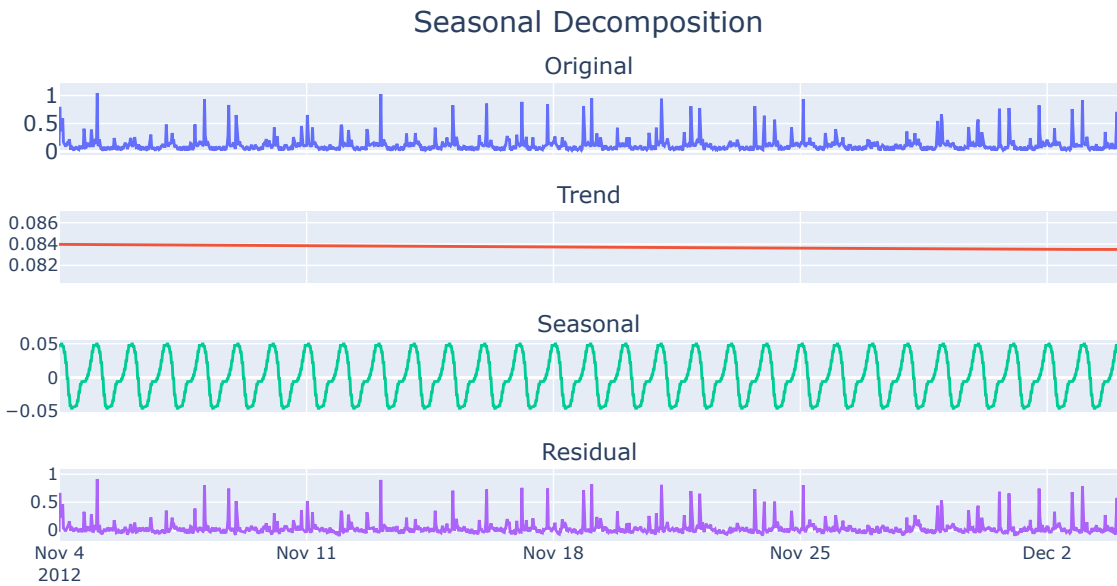
Seasonality and Trend Decomposition using Loess and Fourier Terms (Fourier Decomposition)

```
#Doesn't support missing values, and expects a series or datafeame with datetime index
stl = FourierDecomposition(seasonality_period="hour", model = "additive", n_fourier_terms=5)
res_new = stl.fit(pd.Series(ts.squeeze(), index=ts_df.index))
```

```
fig = res_new.plot(interactive=True)
fig
```



```
fig.update_xaxes(type="date", range=["2012-11-4", "2012-12-4"])
fig
```

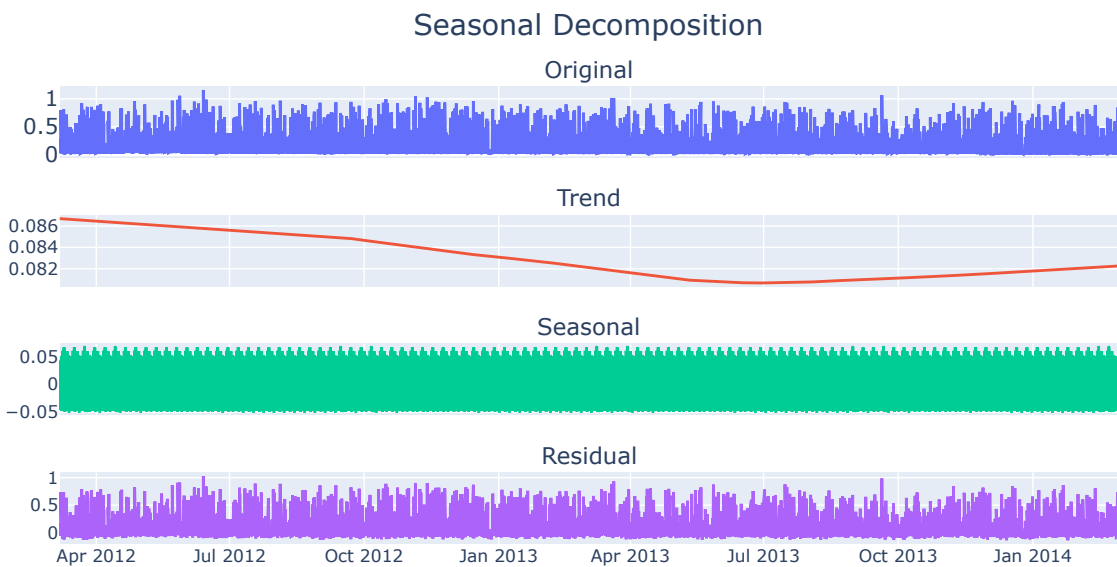


Custom Seasonality

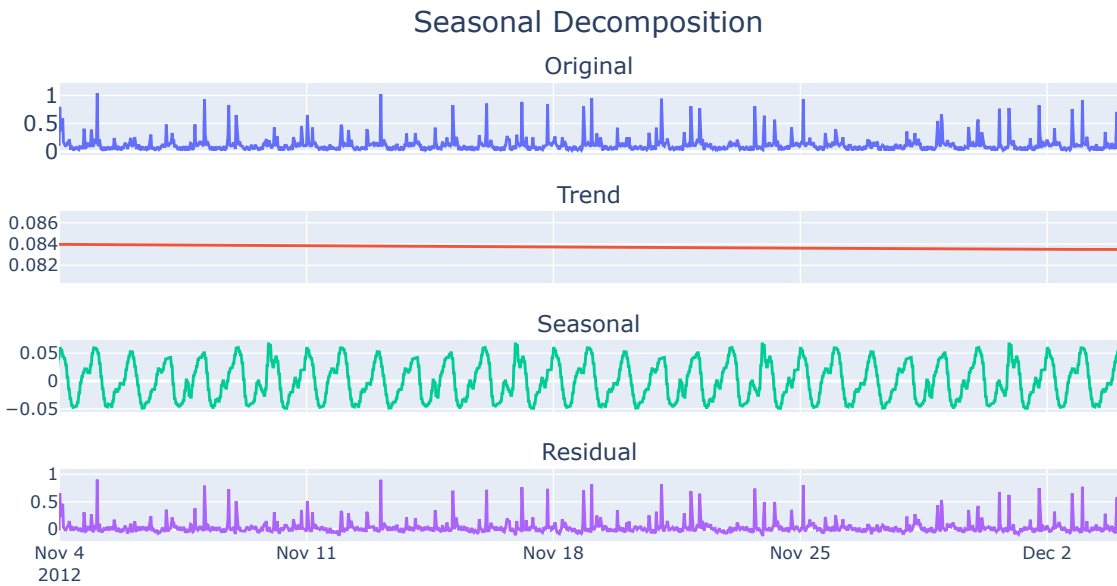
```
#Making a custom seasonality term
ts_df["dayofweek"] = ts_df.index.dayofweek
ts_df["hour"] = ts_df.index.hour
#Creating a sorted unique combination df
map_df = ts_df[["dayofweek","hour"]].drop_duplicates().sort_values(["dayofweek", "hour"])
# Assigning an ordinal variable to capture the order
map_df["map"] = np.arange(1, len(map_df)+1)
# mapping the oprdinal mapping back to the original df and getting the seasonality array
seasonality = ts_df.merge(map_df, on=["dayofweek","hour"], how='left', validate="many_to_one")['map']
```

```
stl = FourierDecomposition(model = "additive", n_fourier_terms=50)
res_new = stl.fit(pd.Series(ts, index=ts_df.index), seasonality=seasonality)
```

```
fig = res_new.plot(interactive=True)
fig
```



```
fig.update_xaxes(type="date", range=["2012-11-4", "2012-12-4"])
fig
```

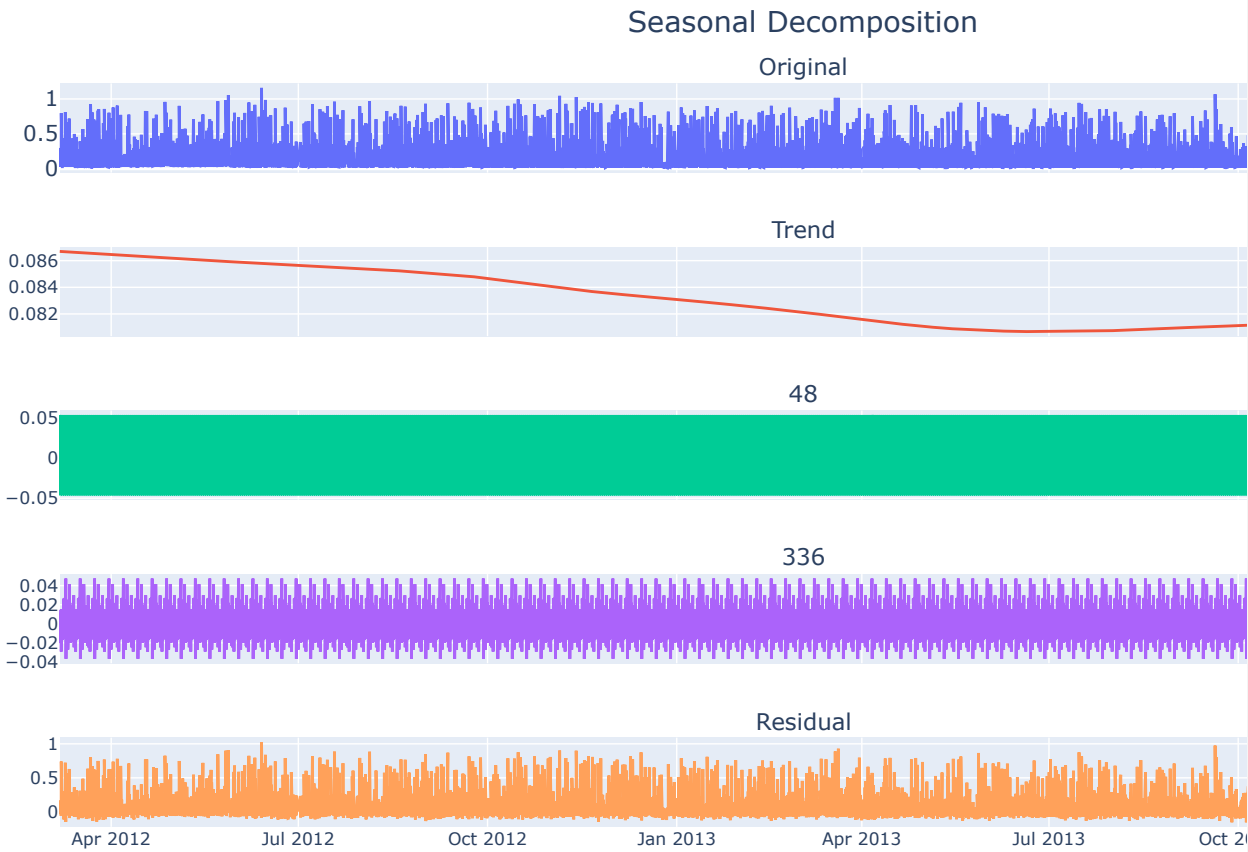


Multiple Seasonality Decomposition using Loess (MSTL)

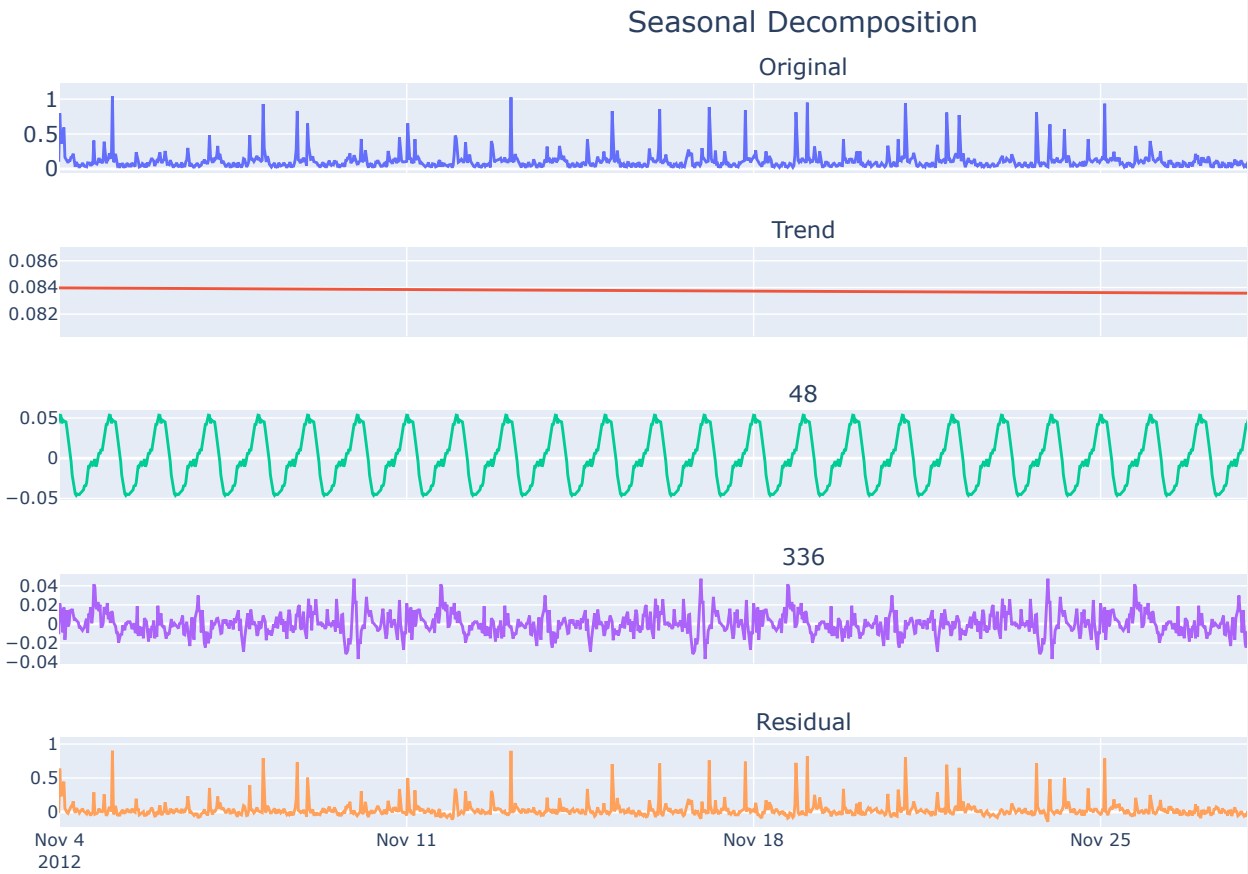
Using Averages as the seasonal model

```
stl = MultiSeasonalDecomposition(seasonal_model="averages",seasonality_periods=[48*7, 48], model = "additiv
res_new = stl.fit(pd.Series(ts, index=ts_df.index))
```

```
fig = res_new.plot(interactive=True)
fig
```



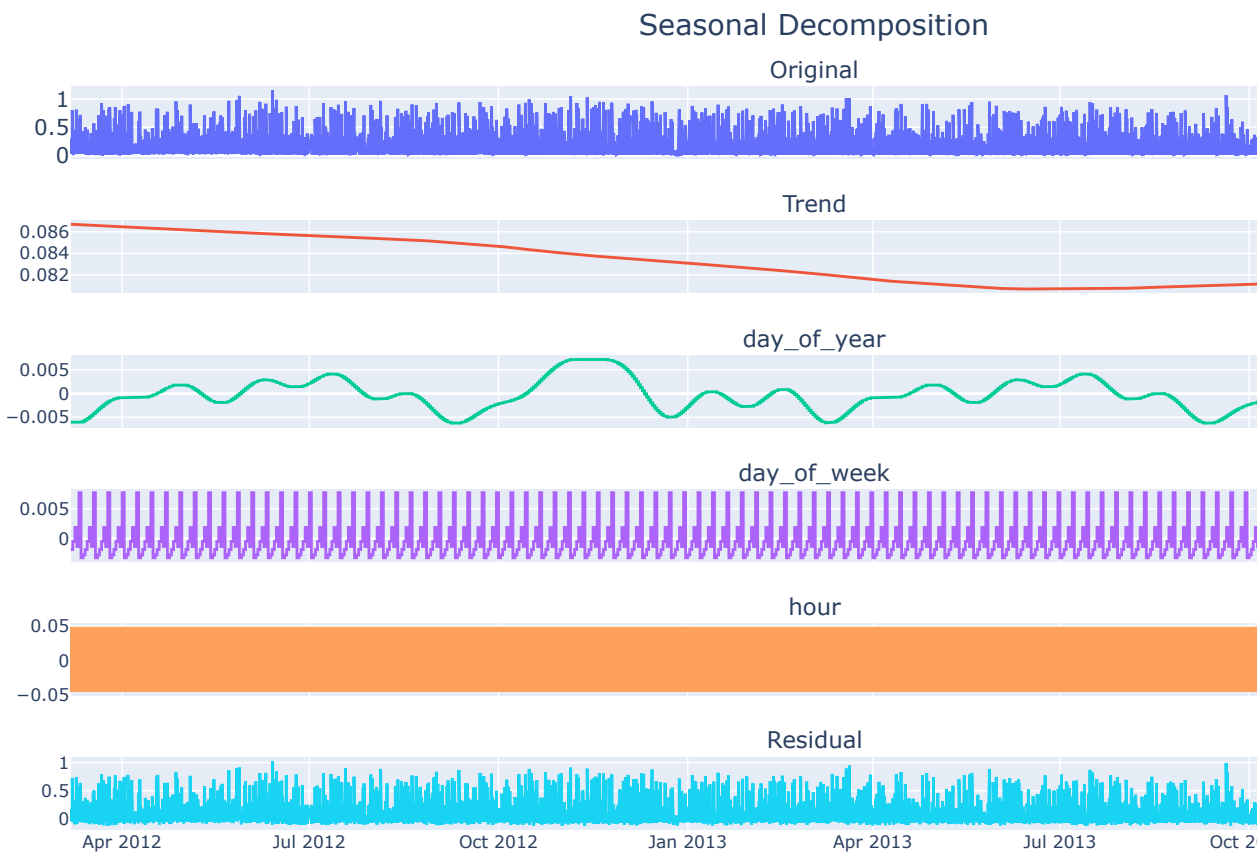
```
fig.update_xaxes(type="date", range=["2012-11-4", "2012-12-4"])
fig
```



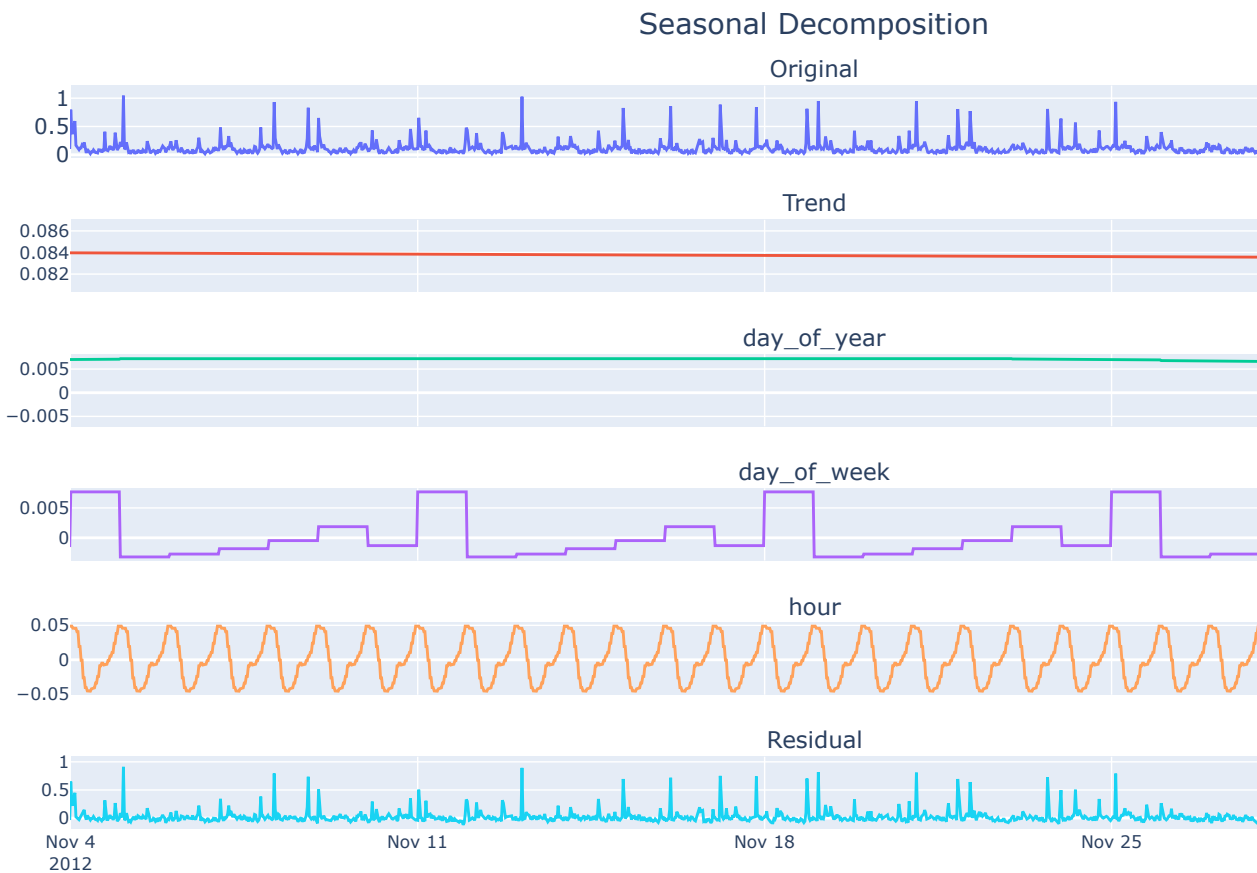
Using Fourier Decomposition as seasonal model

```
stl = MultiSeasonalDecomposition(seasonal_model="fourier", seasonality_periods=["day_of_year", "day_of_week"])
res_new = stl.fit(pd.Series(ts, index=ts_df.index))
```

```
fig = res_new.plot(interactive=True)
fig
```



```
fig.update_xaxes(type="date", range=["2012-11-4", "2012-12-4"])
fig
```



MSTL decomposition in STATSMODELS

break out time series up in to daily and weekly seasonal components.

```
exp_block_df.head()
```

```
ts_df_mstl = exp_block_df[exp_block_df.LCLid=="MAC000322"][['timestamp','energy_consumption']].set_index('timestamp')
ts_df_mstl.head()
```

energy_consumption	
timestamp	
2012-03-07 00:00:00	0.125
2012-03-07 00:30:00	0.104
2012-03-07 01:00:00	0.133
2012-03-07 01:30:00	0.145
2012-03-07 02:00:00	0.109

```
stl_kwargs = {"seasonal_deg": 0, }
mstl = MSTL(season_length=[48, 48*7],
            #windows=[101, 101], # Setting this large along with `seasonal_deg=0` will force the seasonali
            stl_kwargs = stl_kwargs,
            )
mstl_result = mstl.fit(pd.Series(ts, index=ts_df_mstl.index))
```

```
mstl_result.model_['trend'].head()
```

trend	
timestamp	
2012-03-07 00:00:00	0.118023
2012-03-07 00:30:00	0.117975
2012-03-07 01:00:00	0.117928
2012-03-07 01:30:00	0.117880
2012-03-07 02:00:00	0.117833

dtype: float64

```
print(mstl_result.model_.keys())
mstl_result.model_['seasonal48'].head()
mstl_result.model_['seasonal336'].head()
mstl_result.model_['remainder'].head()
```